How-To: Agent Client Protocol (ACP)

ragent speaks the **Agent Client Protocol (ACP)** in both directions. ACP is an open, editor-agnostic protocol in which a *client* drives an *agent* over JSON-RPC 2.0, one message per line, on the agent's standard input and output (stdio). ragent can play either role:

- **ACP client** - ragent drives an external coding agent as a **subprocess**: Claude Code, Codex, or Gemini CLI (or any custom ACP server). While such an agent is active, turns are relayed to it over JSON-RPC on its stdio instead of being sent to a local ragent LLM provider (spec `openhands` FR-015, FR-022).
- **ACP server** - ragent **serves** an ACP-capable editor (Zed, VS Code, JetBrains) over JSON-RPC on its own stdio. Each editor session maps onto a local ragent session whose agent-loop events stream back as ACP `session/update` notifications (spec `openhands` FR-022, FR-028). This endpoint is **feature-gated and off by default**.

This document is the practical manual: the transport model, how to register and bind an external agent, complete configuration examples for Claude Code, Codex, and Gemini CLI, how to stand up the server for an editor, failure handling, and troubleshooting.

For the config key reference see `config.md` section 7.37. For the requirements see `specs/openhands/SPEC.md` FR-015, FR-022, FR-028, FR-036.

Table of Contents

- 1. Concepts
- 2. The ACP Client
  - 2.1 The turn lifecycle
  - 2.2 Binding an agent to an ACP agent
  - 2.3 Streamed updates
  - 2.4 Failure handling
- 3. Worked Examples
  - 3.1 Claude Code
  - 3.2 Codex
  - 3.3 Gemini CLI
  - 3.4 A custom ACP server
- 4. The ACP Server
  - 4.1 Enabling the endpoint
  - 4.2 Requests handled
  - 4.3 Streamed updates
  - 4.4 Wiring an editor
- 5. Configuration Reference
- 6. Troubleshooting
- 7. See Also

---

1. Concepts

Term	Meaning
ACP	Agent Client Protocol: JSON-RPC 2.0, one frame per line, over stdio. ragent implements protocol version `1`.
ACP agent	An external coding agent (Claude Code, Codex, Gemini CLI, or a custom ACP server) that ragent drives as a subprocess.
ACP session	A conversation. In client mode ragent opens it with `session/new`; in server mode ragent returns a fresh id to the editor.
Turn	One user message (`session/prompt`) and everything the agent streams in reply, ending with a terminal `stopReason`.
`session/update`	A notification the agent streams mid-turn: text chunks, thought chunks, tool calls, or a plan.
Client mode	ragent is the client; the external agent is the agent.
Server mode	ragent is the agent; an editor is the client.

The wire protocol version is a single constant, `ACP_PROTOCOL_VERSION = 1` (`crates/ragent-agent/src/acp/mod.rs`). It is exchanged in the `initialize` handshake; a peer that negotiates a different version is logged as a warning but the exchange continues.

---

2. The ACP Client

`crates/ragent-agent/src/acp/mod.rs` owns the client transport and `crates/ragent-agent/src/session/acp_dispa` bridges its output onto the local event bus.

2.1 The turn lifecycle

Every turn is a **fresh subprocess**, so no state leaks between turns and a crashed agent cannot wedge the session. The entry point the session processor uses is `acp::relay_turn`, which performs the whole sequence and then shuts the subprocess down:

```
spawn      AcpClient::spawn   start `command` (+ args, env) in cwd; pipe stdin/stdout/stderr
initialize initialize         negotiate protocolVersion; advertise clientCapabilities
session/new create_session    open a conversation for the session working directory
session/prompt prompt         relay one turn; render each streamed session/update
(shutdown) shutdown           kill the child and abort the writer/reader/stderr tasks
```

The child is started with `kill_on_drop`, so a failed or dropped client tears the process down. Key transport facts:

- **stdin/stdout carry one JSON-RPC frame per line.** Outgoing frames are serialised through a single writer task so a streamed update and a request reply can never interleave mid-line.
- **stderr is drained** into a bounded 20-line tail, kept for error context. It is never parsed as protocol.
- **The handshake is bounded.** `initialize` and `session/new` each get a 30-second timeout (`HANDSHAKE_TIMEOUT`); both are local subprocess round-trips.
- **Client capabilities.** ragent advertises `fs.readTextFile: false`, `fs.writeTextFile: false`, and `terminal: false`: the external agent does its own filesystem and terminal work; ragent does not service those client-side requests.

- **Cancellation.** A turn polls the session's cancellation flag; on cancel the child is killed and the turn fails with a "cancelled by user" protocol error.
- **A client-side request ragent does not implement** (any inbound request carrying an id) is answered with a JSON-RPC "method not found" so the agent is never left waiting.

2.2 Binding an agent to an ACP agent

Registering an ACP agent does **not** by itself change where turns go. An ordinary agent must be explicitly bound, so a local agent is never silently rerouted to an external process. Resolution (`acp_agent_for`) binds an agent to a registered, **enabled** ACP agent when either:

1. the agent's `options.acp` names a registered ACP agent id, or
2. the agent's own `name` equals a registered ACP agent id.

There is deliberately no fallback to `acp.default_agent` for a normal agent. `default_agent` only supplies a default for the picker/resolution surface, not a silent reroute.

So the two ways to route a turn to Claude Code are:

```
// (a) a custom agent that delegates to the "claude" ACP entry by name
{ "agents": { "claude": { "id": "claude", "command": "claude-code-acp" } } }
// then define a ragent agent named "claude", or:

// (b) any agent whose options name the ACP id
{ "name": "coder", "options": { "acp": "claude" } }
```

When a turn is dispatched and the active agent is bound, `SessionProcessor` relays the turn to the external agent and returns immediately. It does not consult the provider registry, and a relay failure never falls back to a local provider (FR-036).

2.3 Streamed updates

Each `session/update` notification decodes into a typed `AcpUpdate` (`parse_update`) and is published onto the local event bus by `acp_dispatch::publish_update`, so the TUI renders it exactly like a local turn:

ACP `sessionUpdate` kind	`AcpUpdate`	Published event
`agent_message_chunk`	`AgentMessageChunk { text }`	`TextDelta`
`agent_thought_chunk`	`AgentThoughtChunk { text }`	`ReasoningDelta`
`tool_call`	`ToolCall { tool_call_id, title, status }`	`ToolCallStart` (and `ToolCallEnd` when the status is already `completed`/`failed`)
`tool_call_update`	`ToolCallUpdate { tool_call_id, title, status }`	`ToolCallEnd`
`plan`	`Plan { entries }`	`AgentNotice` ("ACP agent proposed a plan with N step(s)")

ACP `sessionUpdate` kind	`AcpUpdate`	Published event
any other	`Other { kind }`	`AgentNotice` (“ACP agent update: ”)

Unknown update kinds are preserved rather than dropped, so a future ACP update is surfaced (as a notice) instead of vanishing. The accumulated text of every `agent_message_chunk` is also persisted as the assistant message for the turn.

2.4 Failure handling

Failure handling is first-class (FR-036). Every failure mode **fails the turn**; the relay never blocks indefinitely waiting for more output. The structured cause is an `AcpError` with an `AcpErrorKind` and, for a process exit, an exit code:

`AcpErrorKind`	Cause
`Spawn`	The agent command could not be started (missing executable, permissions).
`Protocol`	A JSON-RPC error reply, a closed channel, a reply for an unknown id, or a user cancel.
`MalformedFrame`	A line on stdout was not valid JSON-RPC; the reader stops and the turn fails.
`Exited`	The subprocess exited (or closed stdout) before the turn completed; a non-zero exit is reported with its code.
`Timeout`	No terminal frame arrived within the per-turn budget; the subprocess is killed.
`Io`	A write to, or read from, the subprocess failed.

The per-turn budget is `turn_timeout_secs` (default **300s**). On timeout the child is killed, matching the “never hang” rule. After a failed or finished turn `shutdown` kills the child and aborts the writer, reader, and stderr tasks so nothing lingers.

---

3. Worked Examples

All examples live under the `acp` block in `ragent.json` (see `config.md` section 7.37). The block is absent by default, which leaves the client subsystem inert.

The generic shape of a registered agent is:

```
{
  "acp": {
    "default_agent": "claude",          // optional: the default entry for the picker
    "agents": {
      "<id>": {
        "id": "<id>",                   // stable id used to select this agent
        "name": "Display Name",         // optional; defaults to the id
        "enabled": true,                // optional; false retires the entry without deleting it
        "command": "<executable>",      // the ACP agent to spawn
        "args": ["..."],                // optional arguments
        "env": { "KEY": "value" },      // optional environment for the subprocess
        "cwd": "/path/to/project",      // optional working dir; defaults to the session dir
        "turn_timeout_secs": 300        // optional per-turn wall-clock budget
      }
    }
  }
}
```

3.1 Claude Code

Claude Code ships an ACP adapter, `claude-code-acp`. Register it, then bind an agent to it.

```
{
  "acp": {
    "default_agent": "claude",
    "agents": {
      "claude": {
        "id": "claude",
        "name": "Claude Code",
        "command": "claude-code-acp",
        "args": []
      }
    }
  },
  "agents": {
    "claude": { "id": "claude", "name": "Claude Code" }
  }
}
```

With a ragent agent named `claude` present, the `claude` agent binds by the name-equals-id rule. Selecting `/agent claude` and sending a message relays the turn to the Claude Code subprocess; its text, thought, and tool-call updates stream into the TUI, and the assistant text is persisted for the session.

If you would rather keep the `coder` agent and delegate it, drop the second agent and give `coder` an option instead:

```
{ "agents": { "coder": { "id": "coder", "options": { "acp": "claude" } } } }
```

3.2 Codex

Codex is registered the same way under its own id. Use the ACP-capable Codex entry your installation ships and point `command` at it:

```
{
  "acp": {
    "agents": {
      "codex": {
        "id": "codex",
        "name": "Codex",
        "command": "codex-acp",
        "args": [],
        "turn_timeout_secs": 600
      }
    }
  },
  "agents": {
    "codex": { "id": "codex", "name": "Codex" }
  }
}
```

A longer `turn_timeout_secs` suits an agent that does more multi-file work per turn. The budget only bounds a *silent* subprocess; an agent that keeps streaming updates is not cut off.

3.3 Gemini CLI

```
{
  "acp": {
    "agents": {
      "gemini": {
        "id": "gemini",
        "name": "Gemini CLI",
        "command": "gemini",
        "args": ["--acp"]
      }
    }
  },
  "agents": {
    "gemini": { "id": "gemini", "name": "Gemini CLI" }
  }
}
```

`env` values are passed verbatim to the subprocess (the parent environment is also inherited, so an ambient `GEMINI_API_KEY` reaches the agent without being written here). Keep secrets out of `ragent.json`; rely on the inherited environment, or inject them at spawn time via the encrypted-store credential mechanism described in `sandbox-backends.md` for container runs.

3.4 A custom ACP server

Any program that speaks ACP on stdio works. This is also how the integration tests exercise the transport: a tiny Python echo agent registered as an ACP agent. A minimal agent answers `initialize`, `session/new`, and `session/prompt`, streaming one `session/update` then replying with a `stopReason`:

```
{
  "acp": {
    "agents": {
      "echo": {
        "id": "echo",
        "command": "python3",
        "args": ["/path/to/echo_agent.py"]
      }
    }
  }
}
```

```
import sys, json

def send(obj):
    sys.stdout.write(json.dumps(obj) + "\n")
    sys.stdout.flush()

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    req = json.loads(line)
    method = req.get("method")
    rid = req.get("id")
    if method == "initialize":
        send({"jsonrpc": "2.0", "id": rid, "result": {"protocolVersion": 1}})
    elif method == "session/new":
        send({"jsonrpc": "2.0", "id": rid, "result": {"sessionId": "s-1"}})
    elif method == "session/prompt":
        send({"jsonrpc": "2.0", "method": "session/update", "params": {
            "sessionId": "s-1",
            "update": {"sessionUpdate": "agent_message_chunk",
                       "content": {"type": "text", "text": "Hello from ACP"}}}})
        send({"jsonrpc": "2.0", "id": rid, "result": {"stopReason": "end_turn"}})
```

This shape (a line-delimited loop, replying to `initialize` and `session/new`, then streaming one update and a terminal reply per `session/prompt`) is the contract ragent expects from any ACP agent.

---

4. The ACP Server

`crates/ragent-agent/src/acp/server.rs` is the mirror image of the client: instead of ragent driving an external agent, ragent **serves** an ACP-capable editor. The editor speaks JSON-RPC 2.0 over ragent's own stdin/stdout; ragent maps each editor session onto a local ragent session and streams that session's agent-loop events back as `session/update` notifications (FR-022).

4.1 Enabling the endpoint

Two gates must both be open, or the command refuses rather than serving an unauthenticated surface:

1. the binary must be built with the `acp-server` Cargo feature, and
2. `acp.server_enabled` must be `true` in the loaded configuration.

When either gate is closed, `ragent acp-server` prints a diagnostic that names the specific closed gate and exits with a usage error (code 2). The endpoint is reachable only through the `ragent acp-server` subcommand - never through the TUI or `ragent run` - so a plain terminal launch never exposes an editor-facing JSON-RPC surface.

```
{ "acp": { "server_enabled": true, "server_agent": "coder" } }
```

```
cargo build --features acp-server
ragent acp-server            # editor attaches over stdin/stdout
```

`server_agent` selects the local ragent agent the endpoint drives. It resolves `acp.server_agent` -> `acp.default_agent` -> the top-level `defaultAgent` (`Config::acp_server_agent`).

4.2 Requests handled

Request	Behaviour
`initialize`	Negotiates the protocol version (`1`) and advertises agent capabilities: `loadSession: false`; prompt capabilities `image: false`, `audio: false`, `embeddedContext: true`.
`session/new`	Opens a local ragent session rooted at the editor's `cwd` (an absolute existing directory; otherwise the process working directory) and returns a fresh `sessionId`.
`session/prompt`	Runs one ragent turn and streams updates; the reply carries the mapped `stopReason`. Only one turn runs at a time - a second prompt while one is in flight is rejected with a "a turn is already in progress" error.
`session/cancel`	Raises the current turn's cancellation flag.

Robustness rules that keep one bad frame from taking the editor connection down:

- A **malformed request frame** on stdin is logged and skipped.
- An **unknown request method** is answered with a JSON-RPC "method not found".
- Error codes used: `-32603` internal (session creation failed), `-32602` invalid params (unknown ACP session id, or a prompt with no text), `-32000` a turn is already in progress, `-32601` server method not supported.

- On stdin close, an in-flight turn is allowed to finish so its updates and reply are flushed before the endpoint returns.

4.3 Streamed updates

Each local session event is translated into an ACP `session/update` payload (`event_to_update`) and sent on the `acp_session_id`:

Local event	ACP update
`TextDelta`	`agent_message_chunk` with `content: { type: "text", text }`
`ReasoningDelta`	`agent_thought_chunk` with a text content block
`ToolCallStart`	`tool_call` with `toolCallId`, `title`, `status: "in_progress"`
`ToolCallEnd`	`tool_call_update` with `status: "completed"` or `"failed"`
`AgentError`	`agent_message_chunk` carrying `error: <message>`

The terminal `stopReason` is mapped from ragent's `FinishReason`:

ragent `FinishReason`	ACP `stopReason`
`Stop`, `ToolUse`	`end_turn`
`Length`, `Truncation`	`max_tokens`
`ContentFilter`	`refusal`
`Cancelled`	`cancelled`

Because the server subscribes to the event bus *before* the turn starts, the first streamed update is never missed, and it drains any updates queued after the turn's final event so the trailing assistant chunk is not dropped.

4.4 Wiring an editor

Point the editor's ACP "agent command" at the ragent binary with the `acp-server` subcommand. The editor launches it as a subprocess and speaks JSON-RPC over the pipe, exactly as ragent would launch Claude Code. A representative editor setting:

```
{
  "agent_servers": {
    "ragent": {
      "command": "/path/to/ragent",
      "args": ["acp-server"],
      "env": {}
    }
  }
}
```

The editor then issues `initialize`, `session/new`, and `session/prompt`, and sees ragent's local agent loop rendered in the editor's chat pane. The agent it drives is whatever `acp.server_agent` resolves to (`coder` by default).

---

5. Configuration Reference

All ACP configuration lives under the `acp` block (`crates/ragent-config/src/config.rs`, `AcpConfig`). The block is optional and absent by default; a default config does not emit the key.

`AcpConfig`

Field	Type	Default	Description
`default_agent`	string	none	Id of the ACP agent used when none is selected explicitly.
`agents`	object	`{}`	Registered ACP agents keyed by stable id.
`server_enabled`	bool	`false`	Master switch for the ACP **server** endpoint. Off by default; an editor can only attach once opted in.
`server_agent`	string	none	The local ragent agent the server drives. Resolves `server_agent` -> `default_agent` -> top-level `defaultAgent`.

`AcpAgentConfig` (one entry in `agents`)

Field	Type	Default	Description
`id`	string	(required)	Stable identifier used to select this agent.
`name`	string	the id	Human-readable display name.
`enabled`	bool	`true`	Whether this entry is offered and can be bound. `false` retires it without deleting it.
`command`	string	(required)	Executable path or name of the ACP agent to spawn.
`args`	string[]	`[]`	Command-line arguments passed to the agent.
`env`	object	`{}`	Environment variables injected into the agent process.
`cwd`	string	session dir	Working directory for the subprocess. Overrides the session working directory when set.
`turn_timeout_secs`	integer	`300`	Per-turn wall-clock budget bounding a silent subprocess.

Field	Type	Default	Description
`system_prompt`	string	none	Optional instruction context for the agent.

Config helpers

Helper	Returns
`Config::acp_enabled()`	`true` when the `acp` block has at least one enabled agent.
`Config::enabled_acp_agents()`	The enabled agents, sorted by id.
`Config::resolve_acp_agent(selected)`	The selected id when enabled, else `default_agent`, else the first enabled agent.
`Config::acp_server_enabled()`	`true` only when `server_enabled` is set explicitly.
`Config::acp_server_agent()`	The driven agent name (`server_agent` -> `default_agent` -> `defaultAgent`).

A project-level `acp` block merges with the user-global one: a project `server_enabled: true` opts in (it can only turn the endpoint on, never off), `server_agent`/`default_agent` are overridden when present, and other agents are merged by id.

---

6. Troubleshooting

Symptom	Cause	Fix
`ragent acp-server` exits with code 2 mentioning the `acp-server` feature	The binary was built without the feature.	`cargo build --features acp-server`.
`ragent acp-server` exits with code 2 mentioning `acp.server_enabled`	The feature is compiled in but the config key is not set.	Add `"acp": { "server_enabled": true }`.
Turns still go to a local provider after registering an ACP agent	No ragent agent is bound to the ACP entry.	Name the ragent agent the same as the ACP id, or set the agent's `options.acp` to that id.
`ACP agent '...' failed (Spawn): spawning '...' failed: ...`	The `command` is missing, not executable, or not on `PATH`.	Install the ACP adapter or give an absolute `command` path.
`... (Exited): agent exited with status N before completing the turn; stderr: ...`	The external agent crashed; its stderr tail is included.	Read the stderr tail; check the agent's own auth/config.

Symptom	Cause	Fix
`... (MalformedFrame): malformed JSON-RPC frame on stdout: ...`	The subprocess printed a non-JSON line to stdout.	The program is not a real ACP agent, or logs to stdout; send logs to stderr.
`... (Timeout): agent produced no terminal frame within 300s`	The subprocess went silent.	Raise `turn_timeout_secs`, or fix the agent so it replies.
`... (Protocol): turn cancelled by user`	The turn was cancelled.	Expected after a cancel; retry if unintended.
Editor never connects to the server	Server was started without both gates.	Build with `--features acp-server` and set `acp.server_enabled: true`.
Server rejects a prompt with “a turn is already in progress”	A second `session/prompt` arrived mid-turn.	Serialise prompts, or wait for the first turn’s reply.

---

7. See Also

- `config.md` section 7.37 - the `acp` config key reference.
- `sandbox-backends.md` - switchable execution backends and container sandboxing, including credential injection for container runs.
- `custom-agents.md` - defining the ragent agents that bind to ACP agents (`options.acp` and the name-equals-id rule).
- `mcp.md` - the other stdio transport ragent speaks; MCP servers and ACP agents are both subprocesses over stdin/stdout but serve different roles.
- `specs/openhands/SPEC.md` - FR-015, FR-022, FR-028, FR-036.